Cryptography Challenge

Run-length encoding: decode pictures by hand, then teach Python to do it.

Name: _____

Class: _____

Date: _____

Today What we are doing

what we do	booklet
Listen: what is encoding, and why do computers compress data?	
Watch: decode the first picture together	
By hand: shade pictures from their clues	pages 2 to 4 (5 if you finish)
Listen: how could a computer do this for us?	
Spyder: run the starter code, trace a row on paper	pages 6 to 8
Listen: `.append` and a loop inside a loop	
Code: write `decode_row`	page 9
Listen: now a whole picture	
Code: write `decode_picture`, then the big pictures	pages 10 and 11
Listen: the finished product	
Code: how fast is the computer? Stretch tasks	pages 11 and 12
Recap: what you learned today	

How it works Use this booklet your way

- Work at **your own speed**. When your teacher is talking, pens and keyboards down and listen.
- Pages 2 to 5 are **by hand** (pencil). Pages 6 to 12 use **Spyder**. Pages 13 and 14 are for reference.
- **Stuck?** Read Hint 1, try again. Then Hint 2, try again. Then Hint 3. Then ask.
- **Ahead of the class?** Keep going. The stretch tasks are on pages 5 and 12.
- **Missed the talk, or want the ideas again?** Pages 13 and 14 explain encoding, lists, loops and a worked example (the heart).

My progress ☐ by hand (2 to 4) ☐ Task 1 ☐ Task 2 ☐ Task 3 ☐ Task 4 ☐ Task 5 ☐ stretch

PART 1 BY HAND

Warm-up: rows

Clue = blanks, coloured, blanks, . . . Shade the boxes with a pencil.

A Decode the rows

Example: `2, 7, 3` means 2 blank, 7 coloured, 3 blank.

2, 7, 3



Your turn. (0 first = starts coloured)

3, 5, 4

0, 4, 8

1, 2, 2, 2, 5

6, 6

0, 3, 3, 3, 3

B Squash it

Write each string as a clue: count each run, then the letter.

`AAAABBBCCCCD` → `4A 3B 4C 1D`

`WWWWWWBBW` →

`XXXXXXXXXX` →

C Stretch it

`4C 2A 5B` →

`3X 1Y 2Z` →

`1A 1B 1A 1B 1A` →

PART 1 BY HAND

Mystery picture 1

Shade the coloured pixels. Clue = blanks, coloured, blanks, . . .

2, 7, 3											
1, 10, 1											
1, 8, 2, 1											
1, 8, 2, 1											
1, 10, 1											
2, 6, 4											
3, 4, 5											
4, 3, 5											
0, 12											

Tip: a clue that starts with `0` means the row starts coloured. Every row's numbers add up to 12.

What is it? _____

How long did it take? _____ min

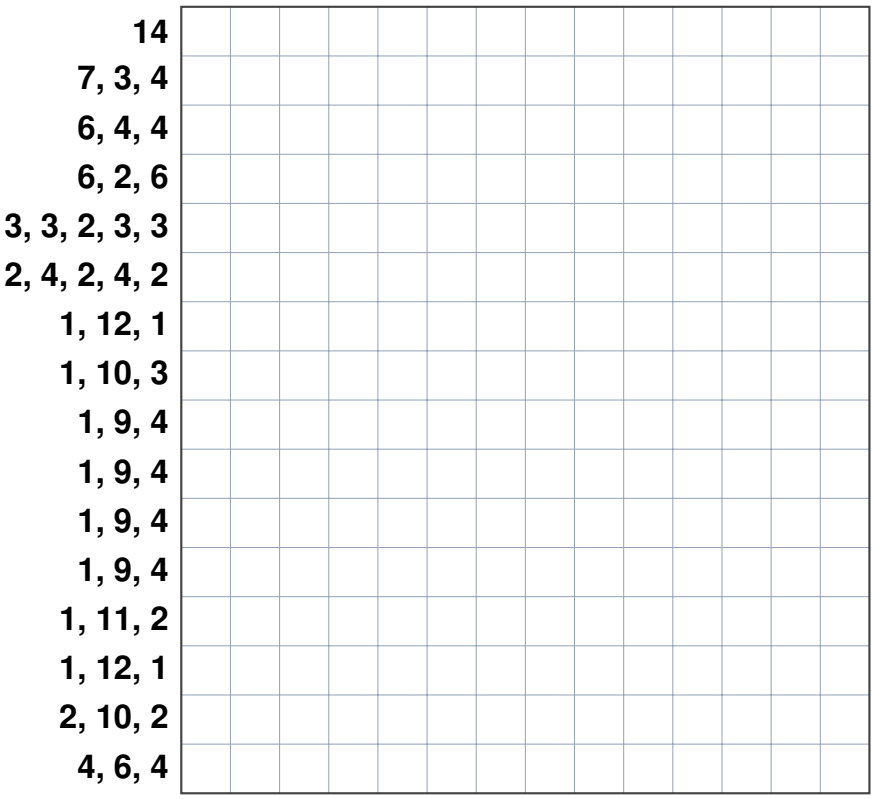
PART 1 BY HAND

Mystery pictures 2 and 3

Do 2 first. Do 3 if you have time. Each row adds up to 14.

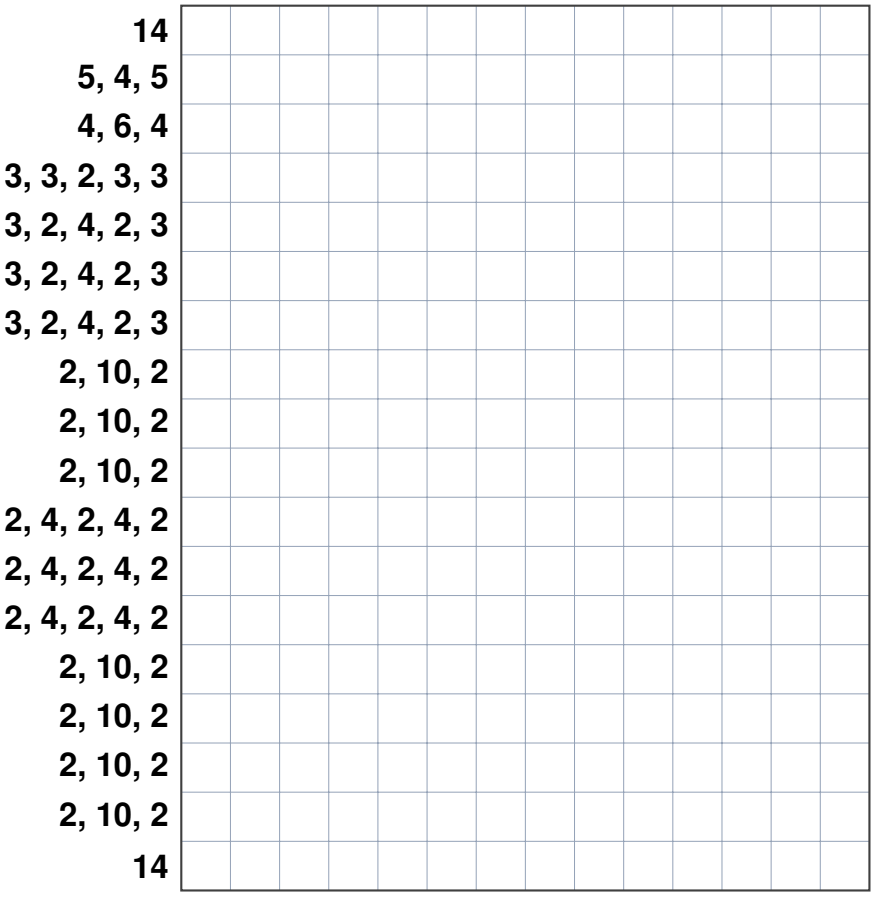
Mystery picture 2

What is it? _____



Mystery picture 3

What is it? _____



if you finish early

Mystery picture 4

Finished early? Shade this one and answer the pattern questions.

14	
4, 6, 4	
3, 8, 3	
2, 10, 2	
1, 12, 1	
1, 12, 1	
1, 3, 1, 4, 1, 3, 1	
1, 12, 1	
1, 12, 1	
1, 12, 1	
1, 5, 2, 5, 1	
1, 4, 4, 4, 1	
14	
14	

? Spot the pattern

1. Which rows have the same clue? Rows _____ and _____

2. Which row is the widest (most coloured pixels)? Row _____

3. What do you notice about the left and right sides of the picture?

PART 2 SPYDER

Get Python ready

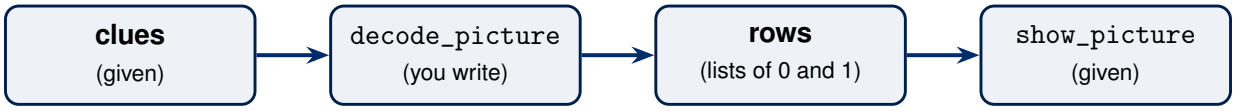
Do each step, then tick the box.

1. **Open Spyder.**
2. **Open the file** `rle_starter.py` . Your teacher will tell you which folder it is in: _____
3. **Check** `pictures.py` is in the *same* folder. You never change that file.
4. **Run it:** press the green triangle (or **F5**).
5. **Look at the Plots pane.** You should see a heart made of dark and blank squares.

☐ **Task 1 done:** I can see the heart.

Remember How the computer sees a picture

- A **row** is a list of 0s and 1s: `row = [0, 0, 1, 1, 1, 0]` . 0 = blank, 1 = coloured.
- A **picture** is a list of rows.
- A **clue** is a list of numbers: blanks, coloured, blanks, . . .
  `[2, 7, 3]` becomes `[0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0]` .



Tasks Your five tasks for today

task	what to do	page	done
1	`rle_starter.py` : run it, see the heart	6	☐
2	finish `decode_row` , test it on `[2, 7, 3]`	9	☐
3	finish `decode_picture` , show the picture from Mystery 1	10	☐
4	`mystery_image_5` , `mystery_image_6` , then `mystery_image_7`	11	☐
5	`mystery_image_7` : how long by hand? How long by computer?	11	☐

PART 2 PAPER FIRST

Be the computer

Follow the code one step at a time. Fill in every empty box.

A Trace `decode_row([2, 7, 3])`

Start: `row = []` `colour = 0`. Step 1 is done for you.

step	number	colour	what gets added	row so far
1	2	0	0, 0	[0, 0]
2	7			
3	3			

B A clue that starts with 0

Trace `decode_row([0, 4, 2])`. Fill in the rest.

step	number	colour	what gets added	row so far
1	0	0		
2	4			
3	2			

? What is different about the first step?

What does `colour = 1 - colour` do? Try `colour = 0`, then `colour = 1`.

PART 2 PLAN

Plan before you type

Write the steps in plain English first. No code yet!

A Steps for `decode_row`

Write the steps the function follows, one per line.

Word bank: start • for each number • add • swap colour • return

1 _____

2 _____

3 _____

4 _____

5 _____

6 _____

B Steps for `decode_picture`

Same again, for a whole picture.

1 _____

2 _____

3 _____

4 _____

Compare with your trace on page 7: does your plan match what you did by hand? Stuck? Page 14 plans the heart the same way.

PART 3 CODE

Task 2: decode_row

Finish the function, run the file, check the test.

```
def decode_row(clue):
    row = []        # the row we are building. It starts empty.
    colour = 0      # we always start with blanks (0)

    # go through each number in the clue (a for loop over the list)
    for ________ in ________:
```

Write your code here

1. add that many copies of colour (hint: range, row.append)
2. swap the colour: 0 becomes 1, 1 becomes 0

```
    return ________      # what do we give back?
```

`print(decode_row([2, 7, 3]))` expected: `[0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0]`

What did YOU get? _____ ☐ it matches

Hint 1: the idea

You need a loop inside a loop (page 13 shows one). The outer loop picks each number from the clue. The inner loop repeats that many times.

Hint 2: the pieces

The inner loop is `for i in range(number):` and each time round it does `row.append(colour)`. After the inner loop finishes, swap the colour: `colour = 1 - colour` (or an if / else).

Hint 3: the shape (fill in the gaps)

```
for number in clue:
    for i in range(______):
        ______________      # add colour to the row
    ______________          # swap the colour (level with the inner for)
```

Nothing printed, or `None`? Check `return row` is at the very end, level with the first `for`.

☐ **Task 2 done:** my `decode_row` gives the right answer for `[2, 7, 3]`.

PART 3 CODE

Task 3: decode_picture

Reuse your row decoder to decode a whole picture.

```
def decode_picture(clues):
    picture = []   # the picture we are building. It starts empty.

    # go through each clue in clues:
    #   1. decode that clue into a row (use your function!)
    #   2. add the row to the picture   hint: picture.append(...)

    return ________
```

Write your code here

Test it: `show_picture(decode_picture(mystery_image_1))` (take the `#` off the line in the starter file)

Does the picture in the Plots pane match your Mystery picture 1? ☐ yes ☐ not yet

Hint 1: the idea

A picture is many rows. You already have a function that turns one clue into one row. Use it once for every clue.

Hint 2: the pieces

Loop over `clues`. Inside the loop: `row = decode_row(clue)`, then `picture.append(row)`.

Hint 3: the shape (fill in the gaps)

```
for clue in ______:
    row = _______________(clue)
    picture._______________(row)
```

Empty picture? Check `return picture` is at the very end, level with the first `for`.

Debugging checklist

- ☐ Did I put `return` at the end, indented once?
- ☐ Is my colour swap INSIDE the outer loop but AFTER the inner loop?
- ☐ Is my `append` indented inside the loop it belongs to?
- ☐ Did I run the file again (F5) after changing it?

☐ **Task 3 done:** my code draws the picture from Mystery 1.

PART 4 BIG PICTURES

Tasks 4 and 5: the finished product

Same code. Much bigger pictures. Then time it.

Task 4 Run your code on the big ones

Take the `#` off these lines in the starter file, one at a time, and run it. Tick each when it works.

```
show_picture(decode_picture(mystery_image_5))  # [ ] worked
show_picture(decode_picture(mystery_image_6))  # [ ] worked
show_picture(decode_picture(mystery_image_7))  # [ ] worked (the one that took ages by hand!)
```

Did your code have to change? ☐ yes ☐ no

Task 5 How long does each way take?

picture	rows	did it work?	time by computer	time by hand
Mystery 1	9			min (page 3)
`mystery_image_5`	47			
`mystery_image_6`	32			
`mystery_image_7`	46			my guess: min

Estimate it. Mystery 1 took me _____ min. `mystery_image_7` has about 5 times as many rows (46 compared with 9), and its rows are longer. So by hand it would take me at least _____ × 5 = _____ min.

Which is faster for the biggest picture, you or Python? Why?

_____

_____

Think: what if the picture was a photo with a million rows? What would you do?

_____

_____

Want to race the computer? Ask your teacher for a big grid to shade by hand while your code runs.

☐ **Task 4 done** ☐ **Task 5 done**

PART 4 STRETCH

stretch

Go backwards: encode_row

Row in, clue out. For when your decoder works and you want more.

```
def encode_row(row):
    clue = []
    # count how many pixels in a row are the same, and add each count to clue.
    return clue

print(encode_row([0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0]))   # should print [2, 7, 3]
```

Write your code here

Hint 1: the idea

Start by counting blanks (0). Keep two variables: `colour` (starts at 0) and `count` (starts at 0). Loop over every pixel in the row.

Hint 2: the pieces

If the pixel is the same as `colour`, do `count += 1`. Otherwise: add `count` to `clue`, change `colour` to the new pixel, and start counting again at 1.

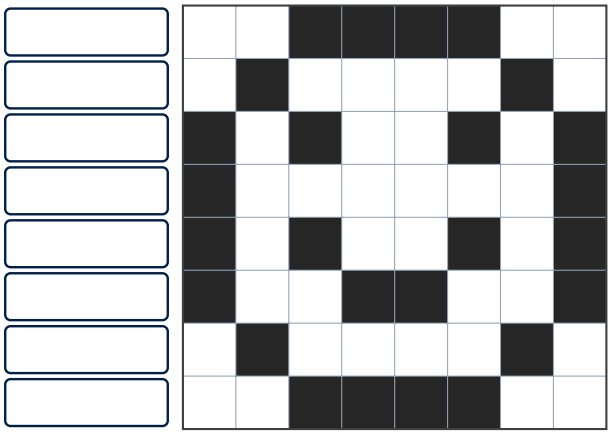
Hint 3: the easy-to-forget bit

When the loop ends, the last run of pixels has not been added to `clue` yet. Add it before you `return`.

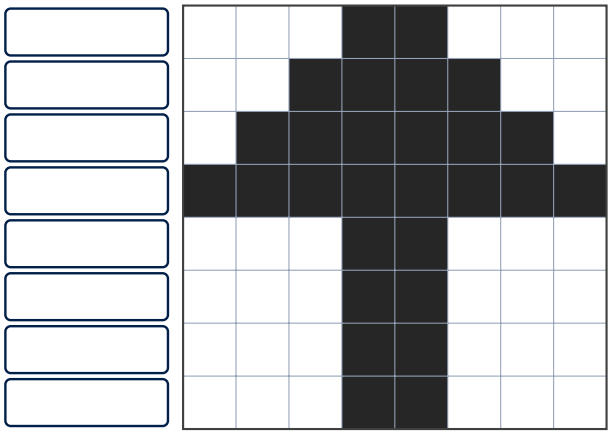
By hand Encode it

Write each row's clue in its box. Blanks first, then coloured (start with `0` if the row starts coloured).

Picture A



Picture B



Think: would clues save more space on a picture with lots of single dots, or with big solid blocks?

Keep going: design your own picture and clues, and swap with a partner. Ask your teacher for the Design your own sheet.

REFERENCE

reference

The ideas you need

Missed the talk? Read this first. Come back to it whenever you are stuck.

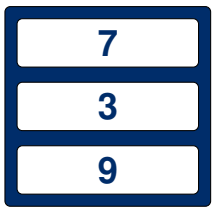
A Encoding, compression and run-length encoding

Encoding means writing the same information a different way (text as numbers, a picture as a grid of numbers). **Compression** is encoding with a goal: say the same thing in fewer symbols. **Run-length encoding (RLE)** stores *what repeats* and *how many times*:

`AAAABBBCCCCD` becomes `4A 3B 4C 1D`

In a picture, every row is blanks, then coloured, then blanks, and so on. So a row of 12 squares can be just three numbers: `[2, 7, 3]` means 2 blank, 7 coloured, 3 blank. It works well on pictures with big blocks of one colour and badly on pictures where every pixel is different.

B Lists: a filing cabinet



A **list** is like a filing cabinet. Each drawer holds one thing, and the drawers are in order.

`numbers = [7, 3, 9]`

A list can hold *other lists*: a filing cabinet whose drawers each hold a smaller filing cabinet.

`grid = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]`

So a **row** is a list of 0s and 1s, and a **picture** is a list of rows (a list of lists).

C .append and a loop inside a loop

`row.append(0)` sticks one more thing on the end of the list `row`. `for number in clue:` does the indented lines once for each number in `clue`. `for i in range(3):` does them 3 times.

```
row = []
for number in [2, 3]:            # outer loop: one pass for each number
    for i in range(number):      # inner loop: repeat that many times
        row.append(0)
print(row)                       # [0, 0, 0, 0, 0]
```

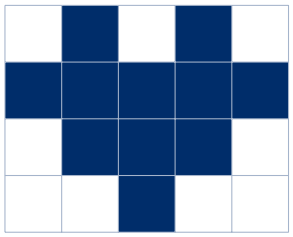
Look for the shape. The inner loop is indented one step more than the outer loop. Anything you want to do once per number (not once per square) goes in the outer loop *after* the inner loop.

reference

Worked example: the heart

One whole decode, planned in plain English and then traced step by step.

A The picture and its clues



row	clue	row as a list
1	`1, 1, 1, 1, 1`	`[0, 1, 0, 1, 0]`
2	`0, 5`	`[1, 1, 1, 1, 1]`
3	`1, 3, 1`	`[0, 1, 1, 1, 0]`
4	`2, 1, 2`	`[0, 0, 1, 0, 0]`

Row 2 starts with `0`: zero blanks, so the row starts coloured. Our job is to go from the clue back to the row.

B Plan it in pseudocode

```
picture = empty list
FOR each clue in the clues:                <- outer loop: one row at a time
    row = empty list
    colour = 0   (blank)
    FOR each number in the clue:           <- inner loop: one number at a time
        add that many of colour to the row
        swap colour   (0 to 1, 1 to 0)
    add the row to the picture
RETURN the picture
```

C Step through row 3: the clue `[1, 3, 1]`

number	colour	add to the row	row so far
start	0	nothing	`[ ]`
1	0	`[0]`	`[0]`
3	1	`[1, 1, 1]`	`[0, 1, 1, 1]`
1	0	`[0]`	`[0, 1, 1, 1, 0]`

Colour is swapped after each number. The finished row is row 3 of the heart. Now the *outer* loop does the same for each of the four clues and adds each row to the picture. **Outer loop: one row at a time. Inner loop: one number at a time.**